



RMI IMPLEMENTATION IN CLOUD ENVIRONMENT

ASSIGNMENT-1



JANUARY 30, 2026

KAVIPRAKASH J
2023115104

Aim

To design and implement a **Remote Method Invocation (RMI) based distributed application** using **Java**, where the **RMI server is hosted in a cloud environment (Microsoft Azure Virtual Machine)** and the **client executes on a local machine**, enabling invocation of remote methods over a network.

Software & Tools Used

Component	Description
Programming Language	Java
Distributed Model	Java RMI
Cloud Platform	Microsoft Azure
Server OS	Ubuntu Linux (Azure VM)
Client OS	Windows 10
JDK Version	OpenJDK 17
Network Port	1009

System Architecture

- RMI Server and Registry run on **Azure VM**
 - Client accesses remote object using **public IP**
 - Communication occurs over TCP/IP
-

Remote Methods Implemented

Method Name	Description
wordCount(String text)	Returns number of words
reverseText(String text)	Returns reversed string
isPalindrome(String text)	Checks palindrome

Execution Steps

Step 1: Install Java on Azure VM

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

Step 2: Compile RMI Programs

```
javac *.java
```

Step 3: Run RMI Server

```
java RMIServer
```

Output:

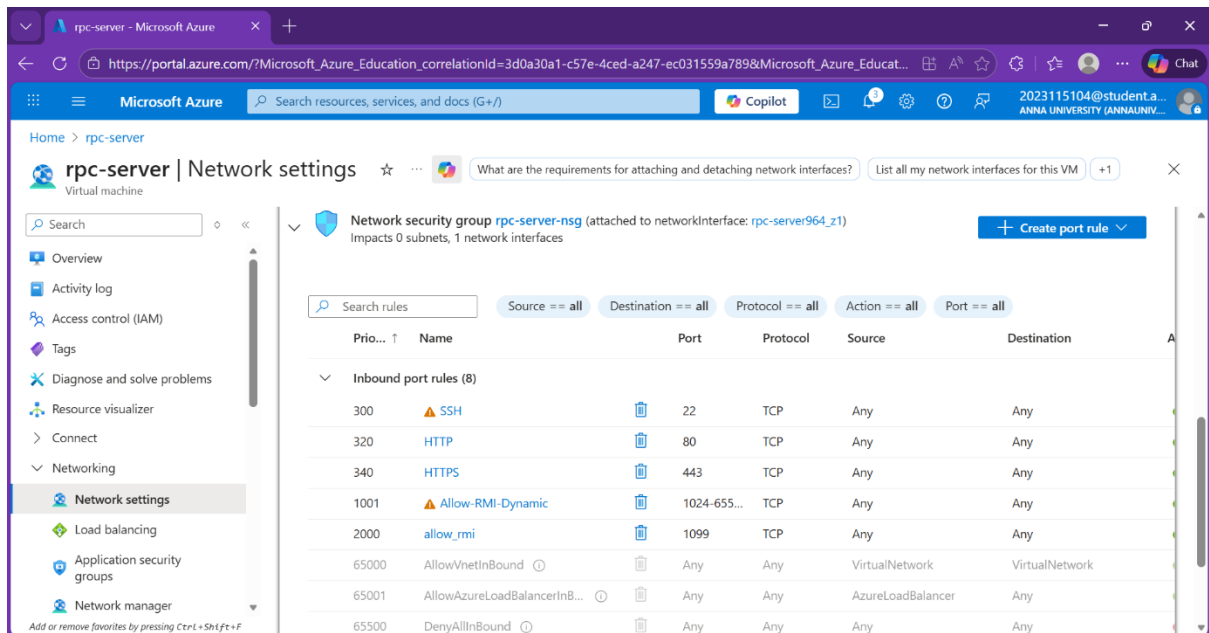
RMI Server running on Azure VM at port **1099**

A terminal window titled 'azureuser@rpc-server: ~' with standard window controls. It shows a series of commands and their outputs: 'nano TextService.java', 'nano TextServiceImpl.java', 'nano RMIServer.java', 'javac *.java', and 'java RMIServer'. The final output is 'RMI Server running on Azure VM (port 1099)'.

```
azureuser@rpc-server: ~  
azureuser@rpc-server:~$ nano TextService.java  
azureuser@rpc-server:~$ nano TextServiceImpl.java  
azureuser@rpc-server:~$ nano RMIServer.java  
azureuser@rpc-server:~$ javac *.java  
azureuser@rpc-server:~$ java RMIServer  
RMI Server running on Azure VM (port 1099)
```

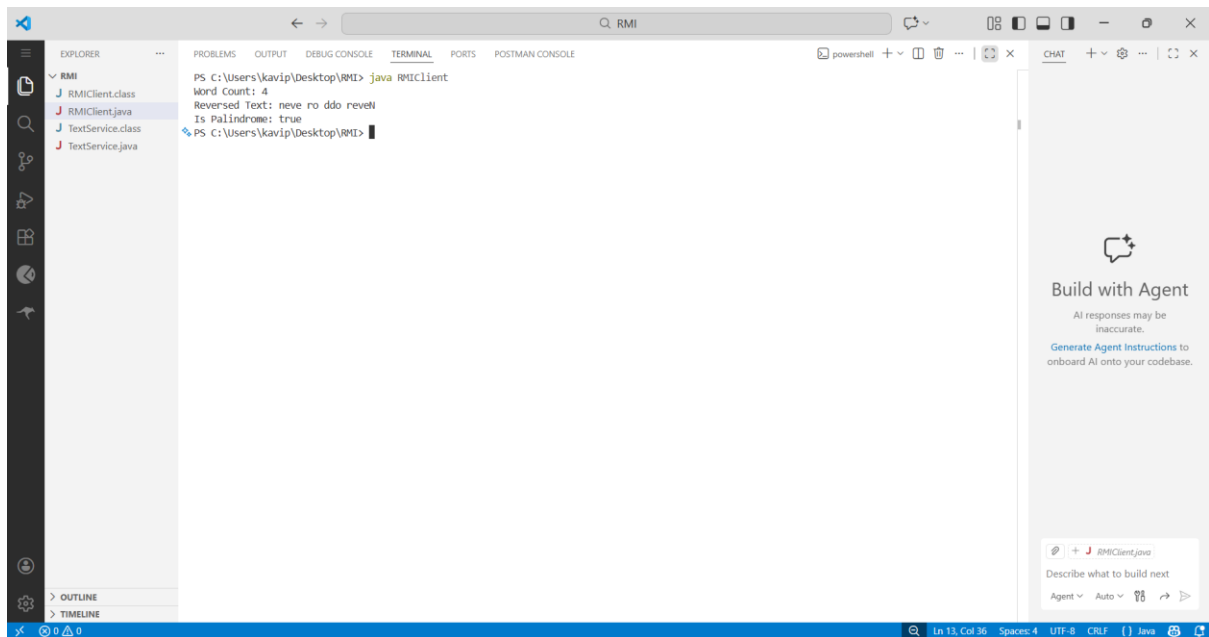
Step 4: Configure Azure NSG

- Allow **Inbound TCP Port 8080**
- Source: Any
- Destination: Any



Step 5: Run Client on Local Machine

java RMIClient



Output

Word Count: 4

Reversed Text: neve ro ddo reveN

Is Palindrome: true

Error Handling

- RemoteException handled in all methods
 - Network timeout handled via try-catch
 - Registry lookup failure handled
-

Result

Thus, a **Java RMI-based distributed application** was successfully implemented and executed in a **cloud environment using Microsoft Azure**, allowing a local client to invoke methods on a remote server and receive correct outputs.

Conclusion

Java RMI provides a robust mechanism for building distributed applications by allowing remote method calls with object-oriented abstraction. Hosting the RMI server on Azure ensures scalability and remote accessibility.
